15.3 队列与并发控制

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~350,000 tokens，输出 ~28,000 tokens（本章合计）

AI 对话系统面临一个独特的并发难题：LLM 调用既贵又耗时（通常 5-60 秒），且同一会话的对话历史必须严格有序。OpenClaw 通过**车道**（Lane）系统实现了精细的并发控制——既保证同一会话的串行性，又允许不同会话和子 Agent 并行执行。

15.3.1 每会话车道（Session Lane）序列化

为什么需要会话级串行化

设想用户快速连续发送三条消息：

```
用户: 帮我写一个排序算法        ← 消息 1
用户: 用 Python                  ← 消息 2（200ms 后）
用户: 要快速排序                 ← 消息 3（100ms 后）
```

如果三条消息并发触发三次 LLM 调用，它们会同时读取相同的对话历史文件，各自独立生成回复，然后尝试同时写入历史——导致数据竞争和对话混乱。

会话车道解决了这个问题：同一会话键的所有请求在同一条车道上排队，严格串行执行。

车道解析

每个 Agent 运行首先解析自己所属的会话车道：

```
// src/agents/pi-embedded-runner/lanes.ts
export function resolveSessionLane(key: string) {
  const cleaned = key.trim() || CommandLane.Main;
  return cleaned.startsWith("session:") ? cleaned : `session:${cleaned}`;
}

export function resolveGlobalLane(lane?: string) {
  const cleaned = lane?.trim();
  return cleaned ? cleaned : CommandLane.Main;
}
```

会话车道名的格式为 `session:{sessionKey}`，例如：

```
session:agent:main:telegram:default:dm:12345
session:agent:work:slack:T01234:dm:U56789
```

每个不同的 `sessionKey` 对应一条独立的车道，确保同一对话的请求互不干扰，而不同对话可以并行处理。

15.3.2 全局车道（Global Lane）限流

除了会话级串行化，OpenClaw 还需要限制全局的 LLM 并发调用数量。原因有三个：

1. **API 速率限制**——大多数 LLM 提供者对并发请求数有限制
2. **资源保护**——每次 LLM 调用消耗大量内存（对话上下文）和网络带宽
3. **成本控制**——并发越高，短时间内消耗的 Token 越多

全局车道类型

OpenClaw 定义了四种全局车道：

```
// src/process/lanes.ts
export const enum CommandLane {
  Main = "main",         // 主车道：用户发起的对话
  Cron = "cron",         // 定时任务车道
  Subagent = "subagent", // 子 Agent 车道
  Nested = "nested",     // 嵌套调用车道
}
```

每种车道有独立的并发限制：

```
// src/gateway/server-lanes.ts
export function applyGatewayLaneConcurrency(cfg: ReturnType<typeof loadConfig>) {
  setCommandLaneConcurrency(CommandLane.Cron, cfg.cron?.maxConcurrentRuns ?? 1);
  setCommandLaneConcurrency(CommandLane.Main, resolveAgentMaxConcurrent(cfg));
  setCommandLaneConcurrency(CommandLane.Subagent, resolveSubagentMaxConcurrent(cfg));
}
```

车道	默认并发数	用途
Main	取决于配置	用户直接对话
Cron	1	定时任务（心跳、提醒等）
Subagent	取决于配置	子 Agent 后台任务
Nested	无限制	嵌套工具调用

两级排队

Agent 运行的排队是**嵌套**的——先排会话车道，再排全局车道：

```
// src/agents/pi-embedded-runner/run.ts
return enqueueSession(() =>       // 先排会话车道
  enqueueGlobal(async () => {     // 再排全局车道
    // ... 实际的 LLM 调用 ...
  }),
);
```

这意味着一个请求需要同时获得两个“通行证”才能执行：会话车道的轮次 + 全局车道的并发名额。

```
请求 A (session-1): — 会话车道排队 ——→ 获得会话轮次 ——→ 全局车道排队 ——→ 执行
请求 B (session-1): — 会话车道排队 (等 A)
请求 C (session-2): — 会话车道排队 ——→ 获得会话轮次 ——→ 全局车道排队 ——→ 执行
```

请求 A 和 C 属于不同会话，可以同时进入全局车道排队（甚至并行执行）。但请求 B 必须等待 A 完成后才能进入全局车道。

15.3.3 队列模式：collect / steer / followup

当 Agent 正忙于处理一个请求时，新消息该怎么办？OpenClaw 提供了多种**队列模式**来处理这种情况：

```
// src/auto-reply/reply/queue/types.ts
export type QueueMode =
  | "steer"           // 转向：注入到正在运行的 Agent 循环中
  | "followup"        // 跟进：排队等待当前运行结束后自动执行
  | "collect"         // 收集：合并为一条消息后执行（默认）
  | "steer-backlog"   // 转向优先，失败则收集
  | "interrupt"       // 中断：终止当前运行，执行新消息
  | "queue";          // 简单排队
```

Collect 模式（默认）

收集模式是最保守的策略。Agent 忙碌期间收到的多条消息会被收集并合并，等当前运行结束后作为一条合并消息执行：

```
用户: 帮我写排序              ← Agent 开始处理
用户: 用 Python               ← 收集（Agent 忙）
用户: 要快速排序              ← 收集（Agent 忙）
Agent: [回复排序]             ← 第一次运行结束
Agent: [处理"用 Python + 要快速排序"]  ← 合并后自动执行
```

Steer 模式

转向模式允许新消息**注入**到正在运行的 Agent 循环中。这利用了 LLM 对话的一个特性——在流式输出过程中，可以追加新的用户消息：

```
// src/agents/pi-embedded-runner/runs.ts
export function queueEmbeddedPiMessage(sessionId: string, text: string): boolean {
  const handle = ACTIVE_EMBEDDED_RUNS.get(sessionId);
  if (!handle || !handle.isStreaming() || handle.isCompacting()) return false;
  void handle.queueMessage(text);
  return true;
}
```

转向只在 Agent 正在流式输出时才有效。如果 Agent 还在等待 LLM 首次响应，转向会失败，回退到收集模式。

Followup 模式

跟进模式将新消息排队，在当前运行结束后**自动触发**一次新的 Agent 循环。这保证每条消息都能得到独立的处理：

```
用户: 帮我写排序              ← Agent 开始处理
用户: 另外，看看我的邮件      ← 排队（独立任务）
Agent: [回复排序]               ← 第一次运行结束
Agent: [自动处理邮件请求]      ← 自动触发第二次运行
```

队列配置

队列模式可以在多个层级配置，优先级从高到低：

```
// src/auto-reply/reply/queue/settings.ts
export function resolveQueueSettings(params: ResolveQueueSettingsParams): QueueSettings
  const resolvedMode =
    params.inlineMode ??                                // 1. 内联指定
    normalizeQueueMode(params.sessionEntry?.queueMode) ?? // 2. 会话级别
    normalizeQueueMode(providerModeRaw) ??             // 3. 通道级别
    normalizeQueueMode(queueCfg?.mode) ??              // 4. 全局配置
    defaultQueueModeForChannel(channelKey);             // 5. 通道默认（collect）
}
```

队列还支持两个重要参数：

参数	默认值	说明
debounceMs	因通道而异	防抖延迟——在此时间内的连续消息合并为一条
cap	配置决定	队列上限——超过后丢弃旧消息或新消息

15.3.4 车道实现（src/process/command-queue.ts）

车道系统的核心实现是一个轻量级的进程内队列：

数据结构

```typescript
// src/process/command-queue.ts
type QueueEntry = {
  task: () => Promise<unknown>;
  resolve: (value: unknown) => void;
  reject: (reason?: unknown) => void;
  enqueuedAt: number;
  warnAfterMs: number;
  onWait?: (waitMs: number, queuedAhead: number) => void;
};

type LaneState = {
  lane: string;
  queue: QueueEntry[];   // 等待中的任务
  active: number;        // 当前活跃任务数
  maxConcurrent: number; // 最大并发数
  draining: boolean;     // 是否正在排水
};

const lanes = new Map<string, LaneState>();
```

每条车道维护一个 FIFO 队列、一个活跃计数器和一个最大并发数。会话车道的 `maxConcurrent` 始终为 1（串行），全局车道的 `maxConcurrent` 由配置决定。

排水泵（Drain Pump）

队列的核心调度算法是一个**排水泵**：

```ts
// src/process/command-queue.ts
function drainLane(lane: string) {
  const state = getLaneState(lane);
  if (state.draining) return;
  state.draining = true;

  const pump = () => {
    // 只要有空闲并发名额且队列不为空，就取出任务执行
    while (state.active < state.maxConcurrent && state.queue.length > 0) {
      const entry = state.queue.shift()!;
      state.active += 1;
      void (async () => {
        try {
          const result = await entry.task();
          state.active -= 1;
          pump();  // 递归：任务完成后立即尝试取下一个
          entry.resolve(result);
        } catch (err) {
          state.active -= 1;
          pump();  // 即使失败也继续排水
          entry.reject(err);
        }
      })();
    }
    state.draining = false;
  };

  pump();
}
```

> **衍生解释：排水泵**（Drain Pump）是一种常见的异步队列调度模式。它的核心思想是：每当一个任务完成（或新任务入队），就尝试从队列中取出尽可能多的任务并发执行，直到达到并发上限或队列为空。这种模式避免了定时轮询的开销，实现了事件驱动的高效调度。

入队与超时警告

```
// src/process/command-queue.ts
export function enqueueCommandInLane<T>(
  lane: string,
  task: () => Promise<T>,
  opts?: { warnAfterMs?: number; onWait?: (waitMs: number, queuedAhead: number) => void
): Promise<T> {
  const state = getLaneState(lane);
  return new Promise<T>((resolve, reject) => {
    state.queue.push({
      task: () => task(),
      resolve: (value) => resolve(value as T),
      reject,
      enqueuedAt: Date.now(),
      warnAfterMs: opts?.warnAfterMs ?? 2_000,
      onWait: opts?.onWait,
    });
    drainLane(lane);  // 入队后立即尝试排水
  });
}
```

每个入队的任务都记录了入队时间和警告阈值。当任务终于出队执行时，如果等待时间超过 `warnAfterMs`（默认 2 秒），系统会输出诊断警告。这对于发现瓶颈非常有用——如果某条车道的等待时间持续很长，说明该车道的并发限制可能需要调高。

辅助查询

```
// src/process/command-queue.ts
export function getQueueSize(lane: string = CommandLane.Main) {
  const state = lanes.get(lane);
  return state ? state.queue.length + state.active : 0;
}

export function getTotalQueueSize() {
  let total = 0;
  for (const s of lanes.values()) {
    total += s.queue.length + s.active;
  }
  return total;
}
```

`getQueueSize` 返回的是等待中 + 活跃中的总数（而非仅等待中的数量），这使得它更准确地反映了系统的当前负载。

本节小结

1. **会话车道**保证同一会话的请求严格串行执行，防止对话历史的并发写冲突。每个 `sessionKey` 对应一条独立的车道。
2. **全局车道**限制整个系统的 LLM 并发调用数量，分为 Main、Cron、Subagent、Nested 四种，各有独立的并发配额。
3. **两级排队**（会话 + 全局）确保请求需要同时获得会话轮次和全局并发名额才能执行。
4. **六种队列模式**（collect/steer/followup/steer-backlog/interrupt/queue）提供了灵活的并发消息处理策略，默认使用 collect 模式合并消息。
5. **排水泵**算法实现了事件驱动的高效调度，每当任务完成或新任务入队时自动尝试执行更多任务。